Pillar 1: Encapsulation — guard the data

Encapsulation means making fields private and exposing controlled access through public methods. The class becomes a capsule: data inside, a defined interface outside.

```
public class Wallet {
   private double balance;        // nobody outside can touch this directly


   public void deposit(double amount) {
      if (amount <= 0) {
         throw new IllegalArgumentException("Deposit must be positive");
      }
      balance += amount;
   }


   public boolean withdraw(double amount) {
      if (amount <= 0 || amount > balance) {
         return false;          // rule enforced in ONE place
      }
      balance -= amount;
      return true;
   }


   public double getBalance() {
      return balance;
   }
}
```

Because balance is private, the only paths to it run through deposit and withdraw — and those methods enforce the business rules. No code anywhere in the application can set a

negative balance, because no code can bypass the checks. That is encapsulation's real payoff: invariants enforced in one place instead of trusted everywhere.

Compare that with a public field: any of a thousand lines could write wallet.balance = -500; and the bug could originate anywhere. The getters-and-setters convention is the visible surface of this idea; the reasoning behind it is covered in depth in **Encapsulation in Java**.

Common mistake: A common mistake beginners make is declaring fields private and then generating public getters and setters for every one of them without thought. A setter with no validation is barely better than a public field. Add a setter only when outside code genuinely needs to change that value, and validate inside it.

Pillar 2: Inheritance — reuse and extend

Inheritance lets a class acquire the fields and methods of another using extends. The child (subclass) reuses the parent's code and adds or specializes behaviour.

```
class Payment {
  protected double amount;

  Payment(double amount) {
    this.amount = amount;
  }

  void process() {
    System.out.println("Processing payment of Rs. " + amount);
  }
}

class UpiPayment extends Payment {
  private String upiId;

  UpiPayment(double amount, String upiId) {
```

```
    super(amount);          // call the parent constructor
    this.upiId = upiId;
  }

  @Override
  void process() {
    System.out.println("UPI payment of Rs. " + amount + " via " + upiId);
  }
}
```

UpiPayment gets amount and the constructor chain from Payment and overrides process() with its own version. The relationship to check before you extend anything: IS-A. A UPI payment IS A payment — good. A Car extending Engine fails the test; a car HAS an engine, so it should hold one as a field (composition) instead.

Java permits single inheritance only — one parent per class — to avoid the diamond ambiguity of two parents defining the same method. The class hierarchy rules, super, and constructor chaining get full treatment in **Inheritance in Java**.

Pillar 3: Polymorphism — one call, many behaviours

Polymorphism means "many forms": the same method call produces different behaviour depending on the actual object. This is where inheritance starts paying rent:

```
class CardPayment extends Payment {
  private String cardRef;

  CardPayment(double amount, String cardRef) {
    super(amount);
    this.cardRef = cardRef;
  }

  @Override
```

```
    void process() {
        System.out.println("Card payment of Rs. " + amount + " on " + cardRef);
    }
}

public class PaymentProcessor {
    public static void main(String[] args) {
        Payment[] todays = {
            new UpiPayment(1500, "ravi@upi"),
            new CardPayment(4200, "HDFC-4321"),
            new UpiPayment(300, "divya@upi")
        };

        for (Payment p : todays) {
            p.process();    // Java picks the right version at RUNTIME
        }
    }
}
```

The loop variable p is typed as Payment, yet each p.process() call runs the subclass's override — UPI logic for UPI objects, card logic for card objects. The JVM decides at runtime by looking at the actual object, not the reference type. This is runtime polymorphism (dynamic method dispatch).

The consequence is huge: PaymentProcessor never needs to know which payment types exist. Add WalletPayment next month, and this loop handles it without a single change. Code that depends on the parent type keeps working as the family grows.

Java also has compile-time polymorphism — method overloading, where one class has several methods with the same name but different parameters. The compile-time vs

runtime distinction is the single most-asked OOP interview comparison; see **Polymorphism in Java** for the full breakdown.

Interview note: When asked to explain polymorphism, skip the dictionary definition and give the two-liner with an example: "Overloading — same name, different parameters, resolved at compile time. Overriding — subclass redefines a parent method, resolved at runtime from the actual object." Then offer the Payment[] loop as proof you have used it. That structure answers the question and the usual follow-up in one go.

Pillar 4: Abstraction — expose what, hide how

Abstraction means programming against what an object does while hiding how it does it. Java provides two tools: abstract classes and interfaces.

```
interface Notifier {
    void send(String to, String message);   // the WHAT — no body
}


class EmailNotifier implements Notifier {
    public void send(String to, String message) {
        // SMTP handshake, MIME headers, retries... the HOW lives here
        System.out.println("Email to " + to + ": " + message);
    }
}


class SmsNotifier implements Notifier {
    public void send(String to, String message) {
        System.out.println("SMS to " + to + ": " + message);
    }
}
```

Code that needs to notify someone depends only on Notifier. It neither knows nor cares whether SMTP or an SMS gateway sits behind send. Swap implementations, add a WhatsAppNotifier, stub it out in tests — the calling code never changes.

You already consume abstraction daily: when you call list.sort(), you use the sorting contract without reading the sort algorithm. Abstract classes (partial implementation, for related classes) versus interfaces (pure contract, any class can join) is a classic exam topic — **Abstraction in Java** covers when to pick which.

Note the division of labour: abstraction hides implementation complexity at the design level; encapsulation hides data at the field level. Interviewers love asking for exactly that distinction.

The four pillars working together

Run the thread through the payments example and you can narrate OOP as one story instead of four definitions:

Pillar	**In the example**	**One-line j**
Encapsulation	Wallet guards balance	Protect da
Inheritance	UpiPayment extends Payment	Reuse and
Polymorphism	p.process() picks the override	One call, r
Abstraction	Notifier hides SMTP details	Depend o

Pro tip: Prepare one personal example like this before interviews — a mini-project domain you can draw in four boxes. Explaining pillars through code you own is far more convincing than the Animal-Dog-Cat example every other candidate recites.

Two related terms complete the vocabulary. Association describes objects that merely use each other; composition describes ownership, where the part cannot outlive the whole — an Order and its OrderLines. When a design choice arises, experienced developers reach for "composition over inheritance": holding a Notifier as a field is more flexible than extending one, because you can swap it at runtime and combine several. Expect at least one interviewer to probe whether you know when NOT to use inheritance — the IS-A test from earlier is your answer.

Also keep the honest limits in mind. Java is not 100% object-oriented: the eight primitive types exist outside the object world for speed, with wrapper classes like Integer bridging the gap. Knowing that nuance, and saying it unprompted, signals you learned Java rather than memorized OOP slides.